

Record Shop Enrichment Pipeline

Optimized Order of Operations

Starting Point: Shop Name + City

Each step feeds the next. The order is not arbitrary — early steps unlock data that later steps depend on. The pipeline is orchestrated by `master_deep_scrape.js`, which runs each scraper in sequence, handles errors gracefully, and supports batch processing with resume capability.

Step 1: Link Discovery via DuckDuckGo

Script: `discover_links.js` **Input:** Shop name + city **Action:** POST to DuckDuckGo HTML search for `"{shop name} {city}"` **Output:** Categorized URLs written to DB:

- `website` — the shop's own domain
- `yelp_url` — Yelp business page
- `google_maps_url` — Google Maps listing
- `social_facebook` — Facebook page
- `social_instagram` — Instagram profile
- `social_tiktok` — TikTok account
- TripAdvisor, press mentions, directory listings

Why First: This is the foundation of the entire pipeline. A single well-crafted search query can surface roughly 80% of a shop's online footprint. Every URL gets categorized automatically by domain pattern matching (e.g., `yelp.com/biz/` → `yelp_url`, `facebook.com` → `social_facebook`).

Every subsequent scraper checks whether its required URL exists in the database before running. No Yelp URL? The Yelp scraper skips. No website? The website crawl skips. Discovery unlocks everything downstream.

Critical Detail: After this step completes, the shop record is **re-read from the database** so that all subsequent scrapers see the newly discovered URLs. Without this refresh, downstream scrapers

would still see NULL values. **Rate Limiting:** Uses randomized User-Agent strings. DuckDuckGo occasionally returns 429 (rate limit) responses, requiring retry logic.

Step 2: Social Discovery via Brave Search API

Script: `discover_socials_brave.js` **Input:** Shop name + city + state (from DB) **Action:** Targeted, site-scoped Brave Search API queries:

- `site:yelp.com "{name}" "{city}"`
- `site:facebook.com "{name}" "{city}"`
- `site:discogs.com "{name}"`
- `site:instagram.com "{name}"`

Output: High-confidence social URLs written to DB (only if not already populated by Step 1) **Why Second:** Brave Search is a paid API (rate-limited to ~1 request/second, 2,000 queries/month on the free tier) but is significantly more accurate than DuckDuckGo for site-scoped queries. This step catches what Step 1 missed — especially:

- **Discogs profiles** with non-obvious usernames (e.g., `discogs.com/user/939Records` wouldn't appear in a general search)
- **Facebook pages** with auto-generated URLs (e.g., `facebook.com/p/939-Records-61566799180474/`)
- **Yelp pages** when DuckDuckGo returned a search page instead of the business page

Each result is validated with **name-matching logic:** the shop's name words must appear in the search result's title or URL to prevent false matches. This prevents, for example, a similarly-named shop in a different city from being linked.

Critical Detail: The shop record is refreshed from the DB again after this step completes. **Cost Consideration:** Because this uses a paid API with monthly limits, it runs after the free DuckDuckGo discovery. It only searches for platforms where Step 1 didn't already find a URL, minimizing API usage.

Step 3: Yelp Deep Scrape

Script: `deep_scrape_yelp.js` **Input:** `yelp_url` from Steps 1–2, OR falls back to city-based search **Action:** Scrapes the Yelp business page using a headless browser (Puppeteer) **Output:**

Saved to `content/{shopId}/reviews/yelp_reviews.json` :

- Star rating (1–5)
- Total review count
- Price level (\$–\$\$\$\$)
- Business categories
- Store hours
- Photos
- Individual review text (reviewer name, date, rating, full text)

Why Here: Yelp provides some of the most structured and reliable business data available. In a single scrape, we get:

- **Rating + review count** — instant quality signal
- **Price level** — market positioning
- **Hours** — operational data
- **Categories** — business classification
- **Individual review text** — raw material for the AI review synthesis in Step 9

If no `yelp_url` was found in Steps 1–2, the scraper falls back to searching Yelp by city, though this is less reliable and may match the wrong business.

Technical Note: Yelp actively blocks headless browsers. The scraper uses stealth browser configurations to work around this, but occasional failures are expected.

Step 4: Google Deep Scrape

Script: `deep_scrape_google.js` **Input:** Shop name + city + state **Action:** Google Places API text search → place details request → photo downloads → review extraction **Output:**

- Google rating and review count
- Verified address, phone number, hours
- Business status (operational, temporarily closed, permanently closed)
- Photos — **downloaded immediately** to Supabase storage as permanent URLs
- Individual reviews → saved to `content/{shopId}/reviews/google_reviews.json`

Why Here: Google is the single most authoritative source for physical business data. The address, phone number, and hours from Google are typically the most accurate available. Google reviews

also tend to be higher volume than Yelp, providing more raw material for synthesis. **Critical Detail** — **Permanent Image Storage:** Google Places photo reference URLs expire after approximately 24 hours. We previously stored these expiring URLs directly, which caused all gallery images to break within a day. The fix: `downloadAndStoreImage()` in `lib/common.js` immediately downloads the image data and uploads it to Supabase storage at `shop-logos/gallery/{slug}/{timestamp}_{source}.jpg`, returning a permanent URL. **Coordinates & Geocoding:** Google Places returns precise lat/lng. If the shop's existing coordinates are missing or significantly different (suggesting a geocoding mismatch), Google's coordinates take priority as they represent the verified business location.

Step 5: Website Crawl

Script: `scrape_shop_website.js` **Input:** `website` URL from Steps 1–2 (must NOT be a Yelp or Facebook URL) **Action:** Multi-page crawl of the shop's own website:

- Starts at homepage
- Follows internal links to About, Contact, Events, Hours, Staff pages
- Up to **10 pages**, max **depth 3** from homepage
- 10-second timeout per page

Output: Saved to `content/{shopId}/website/` :

- Schema.org JSON-LD structured data (hours, address, ratings, business type)
- Phone numbers (regex-extracted from page content)
- Email addresses
- Meta descriptions and page titles
- Full page content (text extraction)
- Internal link structure

Why Here (Not Earlier): The website crawl depends on having the actual shop website URL, which comes from the discovery steps. It's positioned after Yelp and Google because those provide structured data reliably, while website scraping is more variable — every shop's site is different.

What Gets Extracted:

- **Phone/email** from page content — the most common contact info source
- **Schema.org JSON-LD** — when present, this is the cleanest structured data (hours, address, ratings)

- **Owner names** from About pages — patterns like "owned by {name}", "founded by {name}", staff pages with "owner" or "proprietor" title
- **Business descriptions and taglines** — from or prominent headings
- **Event info** — in-store performances, Record Store Day plans, signings
- **Specializations** — format focus (vinyl, CD, cassette), genre depth, new vs. used
- **History** — founding date, location changes, ownership transitions

Important Gate: If the `website` field contains a Yelp or Facebook URL (a data quality issue we found in 230 shops and cleaned up), the crawl is skipped. These garbage URLs were purged during our February cleanup, but the check remains as a safeguard. **Content Volume:** A typical crawl produces 40KB+ of extracted content across multiple pages, providing rich context for the AI synthesis in Step 9.

Step 6: Discogs Scrape

Script: `scrape_discogs.js` **Input:** Shop ID (looks up shop name, searches Discogs for matching profiles) **Action:** Searches `discogs.com/user/` and `discogs.com/seller/` endpoints for matching profiles **Output:**

- Seller rating (percentage)
- Total number of ratings
- Marketplace activity level
- Profile URL → stored as `social_discogs`

Why Here: Discogs seller ratings are the single most relevant quality signal for record shops. A shop with 2,000+ ratings and 99%+ positive feedback is almost certainly excellent — and that signal comes from actual vinyl buyers, not general consumers. **Historical Context:** This scraper was completely missing from the original pipeline. During the 939 Records failure analysis (Feb 25, 2026), we discovered that `vinylhub.discogs.com` was in the `SKIP_DOMAINS` list in `discover_links.js`, which meant we were actively ignoring Discogs profiles. 939 Records had 2,171 ratings with 5 stars on Discogs — data our pipeline completely missed. This was identified as the #1 enrichment gap. **DB Schema:** May require new columns: `social_discogs` (TEXT), `discogs_rating` (NUMERIC), `discogs_ratings_count` (INTEGER), or the data can be stored in existing description fields.

Step 7: Instagram Deep Scrape

Script: `scrape_instagram_deep.js` **Input:** `social_instagram` handle (from Steps 1–2)

Conditional: Only runs if an Instagram handle exists in the database. If `social_instagram` is NULL, this step is skipped entirely. **Output:**

- Follower count
- Post count
- Engagement metrics (likes, comments per post)
- Content themes

Why Here: Instagram data is supplementary but valuable for assessing:

- **Community engagement** — a shop with 10K followers and active engagement is a community hub
- **Marketing sophistication** — posting frequency, content quality, hashtag strategy
- **Vibrancy signal** — active Instagram = active business

Technical Note: Instagram heavily restricts scraping. Success rates vary. This is one of the less reliable scrapers in the pipeline.

Step 8: Event Discovery

Script: `discover_events.js` **Input:** Shop ID + all previously scraped content from Steps 3–7

Action: Analyzes website content, social media posts, and review mentions to identify event patterns **Output:**

- Event types (in-store performances, Record Store Day, signings, workshops)
- Event frequency (weekly, monthly, occasional)
- Artist caliber indicators
- Community programming assessment

Why Late in the Pipeline: Events are extracted from content that has already been gathered by earlier steps. The event discoverer reads:

- Website pages (especially Events/Calendar pages from Step 5)
- Review text (mentions of "great live music" or "amazing RSD events" from Steps 3–4)
- Social media posts (event announcements from Step 7)

Running this before the content-gathering steps would result in finding almost no events.

Step 9: Review Synthesis

Script: `summarize_reviews.js` **Input:** ALL data gathered from Steps 3–8:

- `content/{shopId}/reviews/yelp_reviews.json` (from Step 3)
- `content/{shopId}/reviews/google_reviews.json` (from Step 4)
- `content/{shopId}/website/` (from Step 5)
- Facebook data (if available)
- Discogs ratings (from Step 6)
- Reddit mentions (if discovered)
- Press mentions (if discovered in Steps 1–2)

Action: Multi-source AI synthesis using Anthropic Claude or Ollama. The AI reads all gathered review text and content, then produces a unified summary. **Output:** A "Google AI Overview"-quality narrative covering:

- Overall quality assessment
- Key strengths (e.g., "exceptional jazz selection", "knowledgeable staff")
- Weaknesses or common complaints
- Atmosphere and vibe description
- Specializations and unique offerings
- Customer sentiment themes
- Comparison context (how it stands relative to other shops)

Why Near-Last: This is the crown jewel of the enrichment pipeline — but it's entirely dependent on having raw data to synthesize. The AI reads everything gathered across all prior steps and produces a coherent, multi-source narrative that no single review platform can provide.

Running this step before the scrapers would give the AI nothing to work with. Running it after every individual scraper would produce incomplete summaries. It must run after all content gathering is complete.

AI Provider Selection: Uses Anthropic Claude for highest quality, with Ollama (local) as a fallback for cost control during batch processing.

Step 10: Field Extraction

Script: `extract_fields.js` **Input:** All content files from `content/{shopId}/` directories

Action: Reads all scraped content and extracts structured fields to fill empty database columns

Output: Updates to the `shops` table — **only fills NULL fields, never overwrites existing data:**

- **phone** — extracted from website `phoneMatches` (first valid 10-digit US number, formatted as `(XXX) XXX-XXXX`)
- **owner_name** — extracted from About pages using patterns:
 - "owned by {name}"
 - "founded by {name}"
 - "owner: {name}"
 - Staff page entries with "owner" or "proprietor" title
 - Schema.org `founder` or `employee` with role
- **description** — from or homepage tagline

Why Last: This is the cleanup pass — the safety net of the pipeline. Individual scrapers (Google, Yelp) write some fields directly to the DB during their execution. But other valuable data — particularly from the website crawl — gets saved to content files without being persisted to DB columns.

The field extractor reads everything and fills in gaps. For example:

- Google might provide the address and hours
- Yelp might provide the rating and price level
- But the **phone number** found on the shop's Contact page in Step 5 only exists in `content/{shopId}/website/` until this step writes it to `shops.phone`
- The **owner's name** from the About page similarly only gets persisted here

The "fill NULLs only" rule ensures that high-quality data from earlier steps (like a Google-verified phone number) is never overwritten by a less-reliable website scrape.

Post-Pipeline: Validation & Timestamping

After all 10 scraper steps complete, `master_deep_scrape.js` performs final cleanup:

Data Validation via `validateShopData()` :

- Ensures `name`, `city`, `state` are non-empty strings
- Normalizes hours using `lib/normalize_hours.js`
- Validates all URL fields are proper URLs (strips invalid ones to NULL)
- Ensures `enrichment_status` is a valid string (not `[object Object]` — a bug found in 345 records)
- Strips any `[object Object]` string values from any field

Timestamp: Sets `deep_scrape_at` to the current timestamp

Results Summary: Saves complete scraper results (success/fail/skip status, elapsed time per step) to `content/{shopId}/deep_scrape_summary.json`

Batch Rate Limiting: When processing multiple shops, waits 5–10 seconds between shops to avoid overwhelming APIs and getting blocked

Design Principles

1. Discovery First, Scraping Second

You cannot scrape a URL you haven't found yet. Steps 1–2 are pure discovery (finding URLs). Steps 3–8 are pure extraction (scraping those URLs for data). This separation is fundamental.

2. Refresh After Discovery

The shop record is re-read from the database after each discovery step. Without this, a scraper running in Step 4 would still see the shop's pre-discovery state with NULL URLs, even though Step 1 just found and saved them.

3. Graceful Degradation

Every scraper checks its prerequisites before running. A shop with only a name and city (no website, no social links, nothing) still gets value from:

- Google Places search (Step 4) — works with just name + city
- Yelp city-based fallback (Step 3) — searches by city if no direct URL
- Discovery steps (1–2) — may find URLs that enable later steps

No scraper failure is fatal to the pipeline. Each one is wrapped in try/catch, logged, and the pipeline continues.

4. Raw First, Synthesis Last

Gather all raw data (reviews, page content, metrics, ratings) before running AI synthesis. The review synthesis in Step 9 is only as good as the breadth and depth of its inputs. This is why it runs after all content-gathering steps.

5. Extract Last, Fill NULLs Only

The field extractor (Step 10) is the safety net. It catches any valuable data that individual scrapers saved to content files but didn't persist to database columns. The "NULL only" rule prevents lower-confidence extractions from overwriting higher-confidence data.

6. Permanent Storage

External URLs (especially Google Places photo references) expire. Every image is downloaded immediately and uploaded to Supabase storage with a permanent URL. All scraped content is saved locally to `content/{shopId}/` with structured subdirectories, providing a complete audit trail and enabling re-processing without re-scraping.

7. Deduplication at Every Boundary

Before inserting any new shop, `findExistingShop()` checks for duplicates using:

- **Normalized name match** — lowercase, strip "the ", strip punctuation — plus same city/state
- **Google Place ID** — if both records have one and they match
- **Coordinate proximity** — lat/lng within 0.001 degrees (~100 meters)

If a match is found, the existing record is updated/merged rather than creating a duplicate.

Known Gaps

Identified during the 939 Records deep-dive (February 25, 2026):

Facebook pages found but not deeply scraped — Facebook's auth wall prevents extracting structured About section data (phone, address, hours) from business pages without login

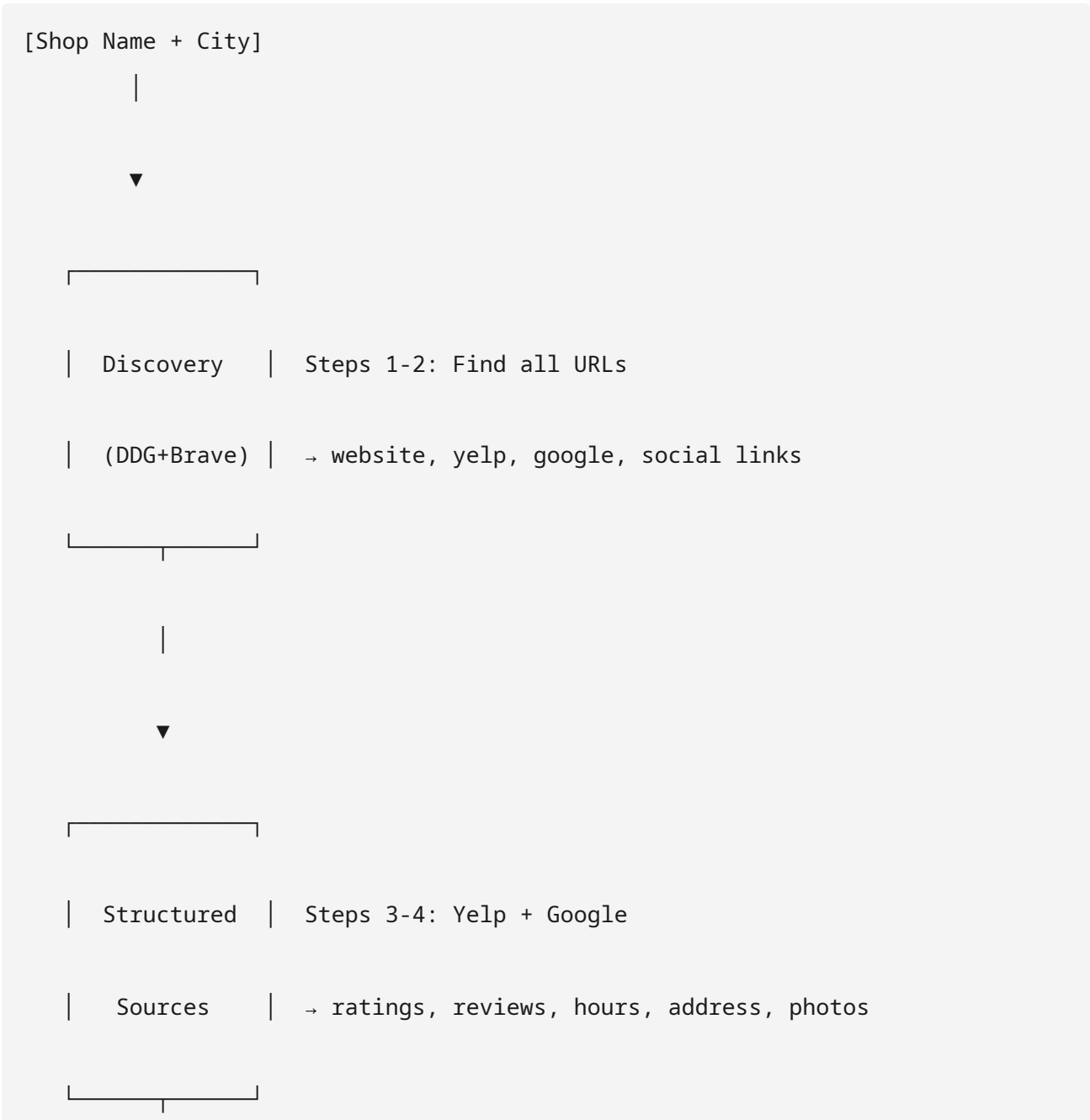
Owner name extraction is pattern-based — misses non-standard About page formats, names mentioned in narrative paragraphs without clear patterns

Discogs scraper is newly built — not yet battle-tested at scale across 867 shops

DuckDuckGo rate limiting — occasional 429 responses require retry logic; no automatic retry currently implemented

No website contact info persistence — the website scraper extracts phones and emails but the pipeline didn't persist them to DB columns until `extract_fields.js` was added as Step 10

Data Flow Summary



|



|

| Website | Step 5: Multi-page crawl

| Crawl | → Schema.org, contacts, about, events

|

|



|

| Specialty | Steps 6-8: Discogs, Instagram, Events

| Sources | → seller ratings, engagement, programming

|

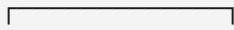
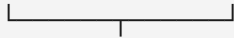
|



|

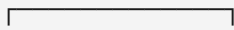
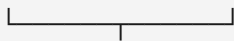
| AI | Step 9: Multi-source synthesis

| Synthesis | → unified review narrative



| Extract | Step 10: Fill empty DB fields

| & Clean | → phone, owner, description (NULLs only)



| Validate | Post-pipeline: data integrity

| & Stamp | → validation, timestamp, summary



Document generated February 27, 2026

Record Shop Enricher — recordshops.us